What Predicts Sprint Spillover Without Hindsight? A Benchmark of Feature Families on Public Jira Data

Hani AbuSharkh*

November 8, 2026

Abstract

Sprint-risk models are usually trained on a present-day Jira export, which shows how a sprint ended rather than what the team knew when it began, and each study reports its own features on its own data. We state formally what a feature may use at prediction time and when an outcome can be measured, and work out four simple consequences that we check against the data. The measurability rule removes three TAWOS projects whose issues are mostly closed without a resolution date. On the remaining 3,676 sprints from 26 projects, scored under one time-ordered protocol for five outcomes, progress counts carry the in-sprint signal. Burndown-shape features rank sprints worse by 0.13 to 0.19 AUC and add nothing when combined with progress counts, across projects and within them. Team history beats committed scope for serious spillover by 0.088 AUC, but nearly all of that gain comes from telling teams apart. Within a project it falls to 0.015, with an interval that includes zero, as a binomial model with one completion rate per team implies. That model learns nothing and ranks sprints about as well as gradient boosting, but its probabilities are too extreme. Four new planning-time features add 0.037 AUC for spillover, within projects too, but the gain does not hold on the 19 projects that did not shape our hypotheses. Nothing we tried predicts late closure well, and on ten outside projects we see no sign that the history gains carry over.

Index Terms— Agile software development, data leakage, data quality, Jira, machine learning, mining software repositories, Scrum, sprint risk

1 Introduction

A Scrum team commits to a set of issues at the start of each sprint. Some sprints end with work left over, and some are closed in Jira days after their planned end. Both are visible to everyone once the sprint is over. The useful question is whether they can be seen coming, at planning or early in the sprint, from what the tracker already records.

Several studies have tried [1–5], and industrial work has studied what makes delivery late [6]. Their results are hard to use for three reasons. The first is hindsight. A Jira export is a snapshot taken years later: an issue's sprint field, story points and status are the final values unless the change log is replayed. A model trained on the snapshot can learn from facts that did not exist at planning [7, 8]. The second is measurement. A label such as "work left over" depends on when each issue was finished, and a tracker does not always record that. The third is comparability. Every study brings its own features, labels, data and protocol, so if one paper reports a good AUC with burndown features and another with iteration counts, nobody can tell which kind of signal does the work.

Our earlier work [9] rebuilt the board as it stood on the first day of each sprint for eight TAWOS projects [10]. Here we turn that reconstruction into a benchmark. We extend it to every TAWOS project with enough sprints, check that the outcomes can be measured, and score the feature families side by side on the same sprints, labels and split. The contributions are as follows.

1. *A head-to-head benchmark with fair comparisons* (Sections 5 and 6). Committed scope, team history, four new planning-time features, in-sprint progress counts, burndown-shape features after Pérez Castillo et al. [4], and the iteration counts of Choetkiertikul et al. [1] that a public export supports, compared both as stand-alone sets and by what each adds to the others.

2. *A measurability check* (Section 4). In three TAWOS projects most sprint issues are closed without a resolution date. Removing them changes which of our hypotheses hold, so any study of spillover on TAWOS needs this check.

3. *Pooled and within-project ranking* (Section 6.5). AUC in this literature, ours included, is pooled over projects, so it also rewards telling teams apart. Scored only within projects, the in-sprint findings hold but most of the planning-time history gain disappears. This matters for any tool that a single team would use on its own sprints.

4. *A formal setting* (Section 3). Admissible features and measurable outcomes are defined on the event log, with four elementary consequences: how empty sprints inflate ROC-AUC, the cost ratio a Youden threshold assumes, a binomial model of which spillover label history can help with, and a beta-binomial extension that says when that model's probabilities can be trusted.

5. *Three candidate labels with a verdict on each, a confirmation set, an outside test and a decision analysis.* Of the three new labels, only serious spillover can be ranked

*H. AbuSharkh is with the Higher Colleges of Technology, Al Ain, UAE (e-mail: habusharkh@hct.ac.ae).

beyond committed scope; seriously late and scope churn cannot, and we report them as negative results. Every hypothesis test is repeated on the projects that played no part in writing the hypotheses, with a multiple-testing correction, and the gains are translated into what a warning is worth.

The short answer is that progress counts carry the in-sprint signal and burndown shape adds nothing. At planning time, team history mostly tells teams apart, not a team's good sprints from its bad ones, and most of what a learned model does is estimate one number per team. We see no sign that the history gains carry to another organisation. Our plan and hypotheses are in the first commit of the repository, made before the full run. We report where they failed and mark every analysis added later.

2 Related Work

2.1 Predicting sprint and iteration outcomes

Choetkiertikul et al. [1] predicted how much of an iteration's committed work would be delivered, using iteration-level counts (issues and points at start, added, removed, done, to do) together with issue-level and team features, on open-source projects. The same group predicted risks and delays at the level of single issues [11, 12], and Sprint2Vec [2] learns a sprint embedding from issue text and developer activity. Hearty et al. [3] modelled velocity in XP with a dynamic Bayesian network. In industry, Kula et al. studied the factors behind on-time delivery in large-scale agile development [6] and predicted delays of user stories from team dynamics [13]. More recently, Pérez Castillo et al. [4] graded finished sprints from eleven burndown-shape indicators (R1 to R11) computed in their Worki tool, and Obike et al. [5] scored sprint deliverability with a weighted index. Effort estimation [14–17] is a neighbouring problem: it predicts the size of an issue, not the fate of a sprint.

All of this work rests on issue-tracker data, and public Jira datasets [10, 18, 19] have made it reproducible. They are, however, exports taken long after the fact. The studies above differ in data, labels and validation, and in how strictly features are limited to what was known at prediction time. In cross-validation over iterations, a training fold can hold iterations that happened after the test ones [1]. No study compares the feature families on common data. We do not claim to reproduce any of them. We implement the parts of two feature families that a public export supports and score them on our sprints, so what we measure is the feature families rather than the papers that proposed them.

2.2 Leakage and time-aware evaluation

Leakage, where a model sees information it would not have when deployed, is a known cause of inflated results [7, 8]. In software analytics, the choice of validation technique changes the performance a defect model appears to have [20], random splits mix past and future [21, 22], and labels that are only known later leak into features [23]. Shepperd et al. [24] found that which research group ran a defect-prediction study explained more of the variation in reported performance than the choice of learner. A common protocol on common data is the remedy we apply here. Our earlier work [9] showed, on eight TAWOS projects, that two reproduced leakage mistakes inflated F1 by up to 0.24 and that shuffling sprints before splitting added 0.07 AUC. The present paper reuses that reconstruction but none of its results.

2.3 Predictive monitoring and warnings as decisions

Predictive process monitoring [25, 26] faces the same problem with event logs: it predicts the outcome of a running case from a prefix of its events, and benchmarks compare methods on common logs [26, 27]. Our in-sprint checkpoints are prefixes in this sense. Alarm-based monitoring [28] treats a warning as a decision with a cost, which is the view we take in Section 3.

2.4 Ranking, thresholds and calibration

ROC-AUC measures ranking [29]. It says nothing about where to set a threshold, and its use for comparing classifiers has known limits [30, 31]. A cost-optimal threshold for a calibrated score follows from the costs alone [32], which is why calibration matters. The Brier score [33] and the calibration slope [34] measure it, and boosted trees are known to produce distorted probabilities unless checked [35].

3 Formal Setting

This section fixes what "no hindsight" means for a Jira export, adds a matching rule for the outcomes, and works out four consequences: how empty sprints inflate ROC-AUC, why a threshold fitted on one group of teams does not carry to another, how the two spillover labels should differ, and why a simple model of sprint completion can rank sprints well yet give probabilities that are too extreme. None of them is deep. Their use is that each turns a pattern that is easy to miss into something the data can confirm or contradict. The first three were in the study plan before the full-cohort run; the fourth (Section 3.6) was added after it, to explain a result.

3.1 The event log and the sprint state

A Jira export is an event log $\mathcal{E} = \{(i, f, v^-, v^+, t)\}$: issue i had field f changed from v^- to v^+ at time t. With each issue's creation time γ_i and resolution time r_i (undefined while the issue is open), the log gives the value of any field at any time by replaying the changes in order. Write $\mathcal{E}_{\le t}$ for the events up to time t, and $\mathcal{F}_t$ for what they tell us together with the sprint calendar, which is fixed at planning. A sprint s has a start a_s, a planned end b_s and a closure time c_s.

The committed set is the board at the start,

$$C_s = \{\, i : s \in \mathrm{sprint}_i(a_s),\ \gamma_i \le a_s,\ r_i \not< a_s \,\}, \qquad (1)$$

where $\mathrm{sprint}_i(t)$ is the issue's sprint field at time t as replayed from $\mathcal{E}_{\le t}$. The outcomes are read off the end state. With $n_s = |C_s|$ and the carry ratio

$$\rho_s = \frac{|\{i \in C_s : r_i > b_s\}|}{n_s}, \tag{2}$$

spillover is $y_s^{\mathrm{sp}} = \mathbf{1}[\rho_s > 0]$, serious spillover is $y_s^{\tau} = \mathbf{1}[\rho_s \ge \tau]$, late closure is $y_s^{\mathrm{late}} = \mathbf{1}[c_s - b_s > \delta]$ with δ one day (three for "seriously late"), and scope churn is $\mathbf{1}[m_s/n_s \ge \theta]$, where m_s counts issues added after a_s and still in the sprint at b_s.

3.2 Admissible features

Definition 1 (Admissible feature)**.** *A feature $x(s,t)$ for sprint s at prediction time t is admissible if it is a function of $\mathcal{F}_t$ alone, that is, of $\mathcal{E}_{\le t}$ and the sprint calendar.*

Two common practices break this rule. First, a history feature that summarises an earlier sprint u of the same project is admissible at a_s only if u's outcome was known by then, $\max(b_u, c_u) \le a_s$. A "previous sprint" picked by start date alone breaks this whenever sprints overlap, which happens for 57% of the sprints in our cohort. Second, a field read from the static export is $\mathcal{F}_\infty$-measurable, not $\mathcal{F}_{a_s}$-measurable. Wherever the log records no change for a field, the replay has to fall back on a value that may postdate planning. We record the share of committed issues affected, ϕ_s, and repeat the main comparisons on sprints with $\phi_s = 0$. Issue type, priority and assignee are left out altogether, because the export only holds their final values.

The outcomes need a rule of their own. The carry ratio uses each committed issue's resolution time r_i. The log records r_i when the issue is resolved, but a tracker can also move an issue into a closed state without recording a resolution time.

Definition 2 (Measurable outcome)**.** *An outcome of sprint s that depends on $\{r_i : i \in C_s\}$ is measurable if, for every $i \in C_s$, either r_i is recorded or the issue is in an open state at the time of export.*

An issue that is closed but has no r_i was finished at an unknown time. Counting it as never finished, which is what a rule that reads only r_i does, makes ρ_s too large and turns a recording habit into spillover. We check this per project (Section 4).

3.3 Empty sprints inflate ROC-AUC

A sprint with $n_s = 0$ cannot spill, so it is a negative by construction. Keeping such sprints in a spillover evaluation rewards the model for something it did not have to learn.

Proposition 1. *Split the negatives of a test set into structural negatives N_0 and ordinary negatives N_1, and let $\pi_0 = |N_0|/(|N_0| + |N_1|)$. Then*

$$\mathrm{AUC}_{\mathrm{all}} = \pi_0\, A_0 + (1 - \pi_0)\, \mathrm{AUC}_{N_1}, \tag{3}$$

where A_0 is the probability that a random positive outscores a random structural negative, and AUC_{N_1} is the AUC on the positives and ordinary negatives alone. If the model ranks every structural negative below every positive, $A_0 = 1$ and the inflation is $\pi_0(1 - \mathrm{AUC}_{N_1})$.

Proof. AUC is the probability that a random positive outscores a random negative. Conditioning on which kind of negative is drawn gives (3). □

The decomposition is elementary and holds exactly, so there is nothing to test. Its use is that it puts a number on the inflation.

To show its size we score spillover on all sprints, empty ones included, with the time split made over all sprints (the one used for the late labels, Section 5). Its test set has 496 negatives, and 430 of them commit nothing, so $\pi_0 = 0.867$. The model (strict+new, gradient boosting) has an AUC of 0.885 on the 688 nonempty sprints of this test set; this differs from Table 4, which uses the split made over nonempty sprints. The measured A_0 is 1.000, so (3) gives 0.985 for the full population, which is the AUC observed there. The gap of 0.099 is entirely the empty sprints. Any study that keeps them in the denominator reports this gap as skill.

3.4 Why a fitted threshold does not travel

Youden's index picks the threshold that maximises sensitivity plus specificity, which is the same as minimising $\mathrm{FNR} + \mathrm{FPR}$. With prevalence π, a miss cost c_{FN} and a false-alarm cost c_{FP}, the expected cost per sprint is $\pi c_{\mathrm{FN}}\mathrm{FNR} + (1 - \pi)c_{\mathrm{FP}}\mathrm{FPR}$. This is proportional to $\mathrm{FNR} + \mathrm{FPR}$ exactly when

$$\frac{c_{\mathrm{FN}}}{c_{\mathrm{FP}}} = \frac{1 - \pi}{\pi}. \tag{4}$$

Proposition 2. *A Youden threshold fitted on a population with prevalence π_{train} is cost-optimal on a population with prevalence π only if the implied cost ratio (4) is the same, that is, only if $\pi = \pi_{\mathrm{train}}$. For a calibrated score the cost-optimal rule warns when $p \ge p^* = c_{\mathrm{FP}}/(c_{\mathrm{FP}} + c_{\mathrm{FN}})$, which does not depend on prevalence [32]. A fixed score threshold carried to a team with lower prevalence therefore acts like a rule that prices a miss at only $(1 - \pi_{\mathrm{train}})/\pi_{\mathrm{train}}$ false alarms.*

The link between a threshold rule and the cost ratio it implies is standard in the ROC literature [31]. What we add is its use to explain why a threshold that worked on TAWOS misses most failures on some new teams. The proposition does not by itself say that sensitivity falls with prevalence; sensitivity depends only on the scores of the positives. It falls in practice because history features make a team's scores track its own base rate, so a team with fewer failures gets lower scores across the board, its failures included.

For late closure the training prevalence is 0.41, so the implied ratio is 1.4. On the 8 testable outside projects the same threshold meets prevalences from 0.07 to 0.59, where (4) would call for ratios from 13 down to 0.7. The observed sensitivity runs from 0.00 to 0.86, with a rank correlation of 0.93 with prevalence. The ranking travels because AUC does not depend on a threshold; the threshold does not travel. A deployment has to recalibrate it per team or set it from stated costs.

3.5 A minimal model of the two spillover labels

Suppose each committed issue of sprint s is finished by the planned end independently with probability q_s, a property of the team and the period rather than of the issue. The number left unfinished is then $\mathrm{Binomial}(n_s, 1-q_s)$, and

$$\Pr[y_s^{\mathrm{sp}} = 1] = 1 - q_s^{n_s}, \tag{5}$$

$$\mathbb{E}[\rho_s] = 1 - q_s, \qquad \mathrm{Var}[\rho_s] = \frac{q_s(1-q_s)}{n_s}. \tag{6}$$

Two predictions follow. By (5) the "any" label is driven by n_s and saturates fast: at the cohort's median carry ratio, 0.67, $q_s \approx 0.33$ and a sprint of three issues already spills with probability 0.963. The median nonempty sprint commits 6 issues, and 90.1% of nonempty sprints spill. So committed scope should carry most of what can be known about plain spillover. By (6) a threshold label on ρ_s is driven by $1-q_s$, the team's completion rate, with n_s entering only through the variance. Serious spillover should therefore depend on planning-time history and capacity rather than on scope, and the gap between scope-only and history-aware models should be larger for it than for plain spillover. Section 6 tests both.

The model also gives a predictor with no learning at all. Estimate q_s from the team's last finished sprints, which is admissible, and read the risk of either label off the binomial distribution. If this one-number-per-team model ranks sprints nearly as well as a learned model, then most of what the learned model does is estimate q_s.

3.6 When issues do not finish independently

We added this part after the full run, when the binomial model ranked well but gave poor probabilities. The binomial model assumes that the issues of a sprint finish independently. They plausibly do not: a blocked dependency, a sick developer or an underestimated epic holds back several issues at once. A standard way to allow for this [36] is to let the unfinished probability p_s itself vary from sprint to sprint around $1-q_s$, with a beta distribution whose spread is set by one parameter $r \in (0,1)$, the correlation between two issues of the same sprint. The count left unfinished is then beta-binomial.

Proposition 3. *If $p_s \sim \mathrm{Beta}\big((1-q_s)(1-r)/r,\ q_s(1-r)/r\big)$ and, given p_s, each committed issue is left unfinished independently with probability p_s, then $\mathbb{E}[\rho_s] = 1-q_s$ and*

$$\mathrm{Var}[\rho_s] = \frac{q_s(1-q_s)}{n_s}\big(1 + (n_s-1)\,r\big). \tag{7}$$

As n_s grows, $\mathrm{Var}[\rho_s] \to r\,q_s(1-q_s)$ rather than to zero, and $\Pr[\rho_s \ge \tau] \to \Pr[p_s \ge \tau]$ rather than to 0 or 1.

Proof. By the law of total variance, $\mathrm{Var}[\rho_s] = \mathbb{E}[p_s(1-p_s)]/n_s + \mathrm{Var}[p_s]$. For the beta distribution above, $\mathrm{Var}[p_s] = r\,q_s(1-q_s)$ and $\mathbb{E}[p_s(1-p_s)] = (1-r)\,q_s(1-q_s)$. Adding the two terms gives (7). Given p_s, $\rho_s \to p_s$ as n_s grows, which gives the limits. □

Two things follow. If $r > 0$, the binomial model's variance is too small by the factor $1+(n_s-1)r$, which grows with the size of the sprint, so the binomial probabilities will be too extreme for large sprints. And committed scope matters even less for serious spillover than the binomial model says, because a large sprint no longer averages its luck away. Section 6 estimates r and checks both. Neither model covers scope added during the sprint. We use them to read the results, not as a claim about how teams work.

3.7 Warnings as decisions

A warning raised at checkpoint c costs κ and, if the sprint would otherwise fail, averts a fraction $e(c)$ of the failure cost F, with e falling as c grows. Warning at calibrated risk p has expected cost $\kappa + Fp(1-e(c))$ against Fp for staying silent, so the warning pays when

$$p \ge \frac{\kappa}{e(c)\,F}. \tag{8}$$

The bar rises as the sprint goes on, because a late warning saves less. Writing $p^* = \kappa/(e(c)F)$ and summing over N sprints, the saving of a warning rule over silence, in units of $e(c)F$ per sprint, is

$$\mathrm{NB}(p^*) = \frac{\mathrm{TP}}{N} - p^*\,\frac{\mathrm{TP}+\mathrm{FP}}{N}, \tag{9}$$

which is the net benefit of decision-curve analysis [37] up to a factor $1-p^*$. At a common threshold, a warning at checkpoint c beats a warning at planning only if $e(c)\,\mathrm{NB}_c > e(0)\,\mathrm{NB}_0$, that is, only if it still averts at least the fraction $\mathrm{NB}_0/\mathrm{NB}_c$ of what an early warning would. Since p^* itself rises as $e(c)$ falls, this is a first-order guide rather than an exact rule. Because (8) needs a calibrated p, we report calibration beside AUC.

4 Data and Reconstruction

4.1 Cohort

TAWOS [10] holds Jira data from open-source projects; 36 of them have sprints. We count a sprint as usable if it is closed, has start, end and completion dates, and neither the end nor the completion is before the start. 32 projects have at least 30 usable sprints. Three of them (MongoDB Compass and the MongoDB C++ and Java drivers) are also in our outside test set, which we pulled from MongoDB's live Jira, so we drop them from TAWOS. That leaves 29 projects.

4.2 Measurability check

We checked the rule of Section 3 per project after the first full run, when three projects showed most of their sprints finishing nothing, and then reran the cohort. The check measures the share of issues ever placed in a sprint whose status is closed, done, resolved or complete but which have no resolution date. Three projects stand out: Atlassian Jira Cloud (69%), Atlassian Confluence Cloud (57%) and Apache MXNet (56%). In every other project the share is at most 0.7%, so any cut-off between 1% and 56% picks the same three projects. In those three projects most sprints appear to finish nothing (Fig. 1), because

closed issues without a date count as never finished. We exclude them. The rule and its cut-off were set before the cohort was rerun, and the cut-off involves no choice, since nothing lies between 1% and 56%. Two other projects, Appcelerator Daemon and Hyperledger Indy Node, also show sprints that finish nothing, but their unfinished issues are genuinely open in the export, so the check gives no reason to drop them; we report the main comparisons without them as a robustness check.

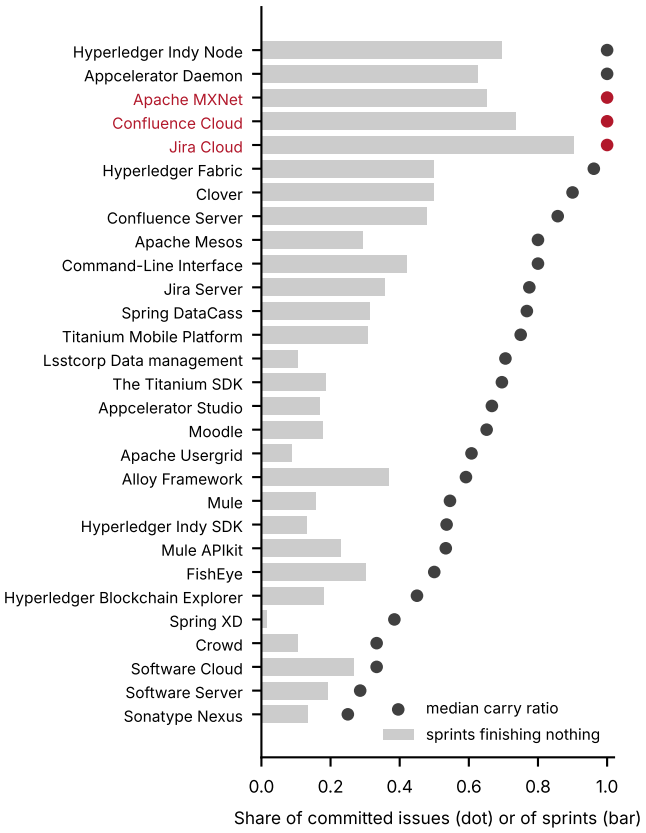


Figure 1: Per project, before the measurability check: median share of committed issues left unfinished at the planned end (dot) and share of sprints that finish none of their committed issues (bar). Red: the three projects where most sprint issues are closed without a resolution date.

The cohort is the remaining 26 projects. After dropping each project's first sprint as a cold start, it has 3,676 sprints from February 2012 to October 2020, between 35 and 557 per project (median 102). Of these, 2,409 commit at least one issue; we call them nonempty. Before the check, 31% of nonempty sprints finished none of their committed issues; after it, 26%.

The outside set is the ten projects used in our earlier study: MariaDB MaxScale and ColumnStore, and eight MongoDB projects, exported from the organisations' public Jira and converted to the TAWOS layout. All ten pass the measurability check. They are used only for testing.

4.3 Reconstruction

For every sprint we replay the Sprint and Story Points fields of every issue back to the sprint's start and form the committed set (1). Issues added later, issues removed later, and their story points at the start all come from the replay. Where an issue has no recorded story-point change, the replay falls back on the exported value; on average 12.9% of a sprint's committed issues are affected. Two custom fields are excluded from the replay: "Actual Story Points" in Spring XD, which is recorded after the work, and a retired duplicate in Moodle.

Reconstruction check. No public record shows the board as it stood at each sprint start, so the replay cannot be compared with ground truth. It can be checked for consistency: replaying an issue's log forward must end in the values the export shows. For story points it does in 99.7% of the 30,520 issues with a story-point history. For the sprint field it does in 99.6% of the 30,401 issues where both the log and the export name a sprint of the cohort. For another 3,455 issues the export shows no sprint although the log's last event names one; the replay follows the log. Of the issues ever placed in a sprint, 79% have a sprint history; for the rest the replay has to take the exported sprint as the sprint at the start, the sprint-field counterpart of the story-point fallback. We repeat the main comparisons on projects where at least 85% of such issues have a history (Section 6.8).

5 Outcomes, Features and Protocol

5.1 Outcomes

Table 1 lists the five outcomes. Late closure and spillover are carried over from our earlier work. The other three are new.

Serious spillover asks how much was left over, not whether anything was. We fixed its primary threshold at a carry ratio of 30% before seeing the full cohort. That was a poor choice: the median nonempty sprint leaves 67% of its committed issues unfinished, so the label is 82% positive, not far from plain spillover (90%). The median-based version (carry ratio above the training-set median, 49% positive), added after the development run, is the balanced one. We keep the primary threshold as planned and report the median version beside it as a secondary result.

Seriously late means the sprint was closed in Jira more than three days after its planned end (seven as a sensitivity check). Like late closure, it records an administrative event, only a larger one.

Scope churn means the issues added after the start and still in the sprint at its planned end make up at least 30% of the committed count (50% as a sensitivity check). It is defined only for sprints with five or more committed issues, and it is predicted at planning time only, because during the sprint the additions so far give the answer away.

5.2 Feature sets

Table 2 lists the sets. Four of them are nested (scope within strict within strict+new within ours), so part of the benchmark is an ablation of one feature set. The two prior families, worki

Table 1: Outcomes, populations and positive rates (26 TAWOS projects). Nonempty: sprints that commit at least one issue. Indented rows are sensitivity versions.

Outcome	Population	Sprints	Positive
Spillover	Nonempty	2,409	90%
Serious spillover	Nonempty	2,409	82%
Late closure	All	3,676	41%
Seriously late	All	3,676	23%
Scope churn	Nonempty, 5+	1,500	66%
Serious sp., median	same	2,409	49%
Serious sp., 20%	same	2,409	86%
Serious sp., 50%	same	2,409	71%
Seriously late, 7 days	same	3,676	13%
Scope churn, 50%	same	1,500	50%

Table 2: Feature sets. Planning-time sets are scored at the sprint start; in-sprint sets at 25% and 50% of the planned duration. †: added after the full run.

Set	Contents
scope	Committed issues, committed story points, sprint length.
strict	scope, parallel sprints, the last three finished sprints (completion ratio, carry-over, delivered points, completed and committed issues), scope relative to capacity, past lateness, and the project.
strict+new	strict and the four new features below.
choet	Iteration counts of [1] that the export supports: duration, issues and points at start, added and removed since start, done and to do at the checkpoint.
choet+ctx†	choet and everything in strict+new, including the project.
worki	Burndown-shape indicators R1 to R11 after [4], on issue counts.
ours	strict+new and 13 progress counts: elapsed share, days left, done and remaining issues and points, burn gap, required and observed rate, added and removed ratios.
ours+worki	ours and R1 to R11.

and choet, are partial versions: they keep what a public export supports. All features are admissible in the sense of Section 3.

The four *new planning-time features* are: the share of committed issues that had been in another sprint before (carried in); the log of the median age of committed issues at the start; the share of committed issues with no estimate; and the share whose estimate was changed before the start (re-estimated).

The *worki* set follows our reading of the verbal definitions in [4], computed on the daily burndown of issue counts up to the checkpoint. Their study graded finished sprints from hours; we score the same kind of indicator during the sprint from issue counts, which is a different task, so a large gap between ours and worki says little on its own. The fair question is whether burndown shape adds anything to progress counts, which ours+worki minus ours answers. The *choet* set leaves out what a public export cannot support: in-progress counts (there is no status history), team size and the Scrum master (the assignee is a final value), and all issue-level features. Ours differs from choet both in planning context (history, new features, project) and in its progress counts. The set choet+ctx separates the two: choet+ctx minus choet is what planning context adds to their counts, and ours minus choet+ctx is what our progress counts add given the same context. Neither comparison ranks the two studies.

5.3 Models and protocol

We use logistic regression with balanced class weights, a random forest [38] (300 trees, depth 5) and histogram gradient boosting [39] (depth 3, learning rate 0.05, 200 iterations), all from scikit-learn [40] with fixed settings and seed 42.

The *primary analysis* is gradient boosting on a per-project time split: the first 70% of each project's sprints by start date for training, the rest for testing. The split is made twice, because the populations differ: over all sprints for the late labels (1,118 test sprints) and over nonempty sprints only for the spillover labels (734 test sprints). AUC is pooled over the test sprints of all projects; Section 6.5 also scores it within projects. For each outcome and comparison we report one paired AUC difference with two 95% bootstrap intervals from 2,000 resamples: one that resamples whole projects [41], which is the test, and one that resamples sprints. For the hypothesis comparisons we also report a two-sided bootstrap p-value from 10,000 project-level resamples and adjust it over all of them with Holm's method [42]. Because AUC can hide poor performance on a rare class [43], we also report average precision for finding the minority class where the positive rate is high. Everything else is secondary: the other two models, leave-one-project-out testing (no project feature), five time-ordered repeats, a tuned model, a single calendar cut-off in place of the per-project split, and the outside projects (trained on all of TAWOS, without the project feature and without story points, which MariaDB does not record). Decision thresholds come from Youden's index [44] on out-of-fold training scores, never from the test set.

5.4 Hypotheses

We wrote the study plan, the primary analysis and four hypotheses into a dated plan before the full-cohort run, and logged every later change with its reason. The plan is in the first commit of the repository. A hypothesis holds only where the project-level interval is above zero.

- **H1, new features** (strict+new minus strict, planning time): positive for scope churn and serious spillover, small and positive for spillover, about zero for the two late labels.
- **H2, burndown shape** (ours minus worki, spillover labels): positive at both checkpoints.
- **H3, planning context** (ours minus choet): positive and below 0.05 for every outcome.
- **H4, new labels** (strict+new minus scope, planning time): positive for seriously late and scope churn, about zero for serious spillover.

H4 and the binomial reading of Section 3 disagree about serious spillover. H4 says history adds nothing beyond scope; the binomial model says history should add more there than for plain spillover. Both were in the plan, and we did not reconcile them before the run. We judge the result against H4, which is the stated hypothesis.

The plan was written after a development run on eight projects. Of these, 7 remain in the cohort and hold 2,138 of its sprints. The other 19 projects (1,538 sprints) had not been looked at when the hypotheses were written, so we repeat every hypothesis test on them alone, training and testing on those projects only, as a confirmation set. The measurability and reconstruction checks, the choet+ctx set, the Holm adjustment, the confirmation set and the within-project analysis were added after the full run, and the plan logs them as such.

6 Results

6.1 The hypotheses at a glance

Table 3 and Fig. 2 give all 22 hypothesis comparisons. Of these, 8 have a project-level interval above zero, 5 survive the Holm adjustment, and 6 hold in both the full cohort and the confirmation set. These counts are less exact than they look. 3 of the intervals, all for H3, end within 0.001 of zero. The one for spillover at 50% clears zero by 0.0002, and with the 10,000 resamples used for the p-values it no longer does, while the one for serious spillover at 50% then just clears it. We count them as the table shows but read none of them as evidence either way.

H1, new features: holds for spillover only, and does not replicate. The new features add 0.037 [0.022, 0.056] for spillover, which survives the Holm adjustment, but only 0.016 [−0.016, 0.048] on the confirmation set. For serious spillover the gain is 0.017 [−0.008, 0.048], with the same sign in all three models. For the two late labels it is about zero, as predicted (for seriously late the interval ends just below zero). Within projects the spillover gain is the same size (Section 6.5). We read H1 as a small gain for spillover that we cannot confirm.

H2, burndown shape: holds everywhere. Ours beats worki by 0.13 to 0.19 AUC on both spillover labels at both checkpoints, in the full cohort and on the confirmation set. Because the two sets were built for different tasks (Section 5), the size of this gap says little by itself; the fair test is in Section 6.3.

H3, planning context: right direction, too weak to hold. Ours beats choet for every outcome, by 0.03 to 0.09, so in 5 of the 8 comparisons the gain is above the 0.05 ceiling we predicted. On the full cohort the project-level interval clears zero clearly only for seriously late at 50%; for the spillover labels it sits on zero. On the confirmation set the gain is larger and holds for both spillover labels. Section 6.3 shows where it comes from.

H4, new labels: fails where it was most specific. We predicted that history would add nothing to serious spillover beyond scope. It adds 0.088 [0.018, 0.161] on the time split, 0.138 [0.088, 0.183] under leave-one-project-out testing and 0.178 on the confirmation set. It does not survive the Holm adjustment, but it holds under leave-one-project-out testing and every robustness variant, and replicates on the confirmation set. All

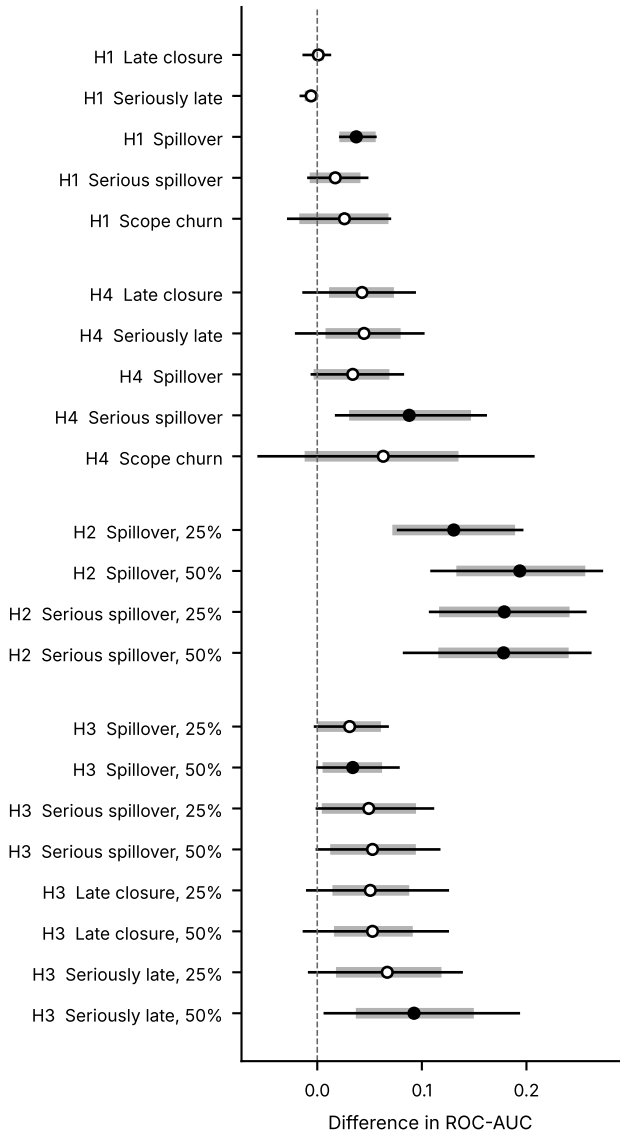


Figure 2: The hypothesis comparisons of Table 3 on the full cohort. Dot: difference in AUC (filled where the project-level interval is above zero). Black line: 95% interval over projects. Grey bar: 95% interval over sprints.

of these scores are pooled over projects, though, and within projects the gain falls to 0.015 [−0.023, 0.088] (Section 6.5). History tells which teams leave more work over, not which of a team's sprints will. Seriously late and scope churn do not hold.

6.2 Planning time

Table 4 gives the AUC of the planning-time sets. Late closure is hard: scope alone reaches 0.682, and the best set 0.725. The binomial model of Section 3 predicts that history should beat scope by more for serious spillover than for plain spillover. It does: strict minus scope is −0.003 for spillover against 0.071 for serious spillover on the time split, and 0.016 against 0.114 under leave-one-project-out testing. Plain spillover is mostly a

Table 3: All hypothesis comparisons (gradient boosting, per-project time split). Δ: difference in AUC with its 95% interval over projects. p_{Holm}: bootstrap p-value from 10,000 project-level resamples, adjusted over the 22 comparisons. Confirmation: the same comparison trained and tested on the 19 projects not used in the development run. ✓: interval above zero.

		26 projects				Confirmation	
	Outcome	Δ	95% interval	p_{Holm}		Δ	
H1	Late closure	+0.001	[−0.013, +0.012]	0.975		−0.009	
H1	Seriously late	−0.006	[−0.016, 0.000]	0.628		+0.002	
H1	Spillover	+0.037	[+0.022, +0.056]	0.006	✓	+0.016	
H1	Serious spillover	+0.017	[−0.008, +0.048]	0.975		+0.035	
H1	Scope churn	+0.026	[−0.028, +0.069]	0.975		+0.006	
H2	Spillover, 25%	+0.131	[+0.077, +0.196]	0.002	✓	+0.158	✓
H2	Spillover, 50%	+0.194	[+0.109, +0.272]	0.002	✓	+0.252	✓
H2	Serious spillover, 25%	+0.179	[+0.108, +0.256]	0.002	✓	+0.236	✓
H2	Serious spillover, 50%	+0.178	[+0.083, +0.261]	0.006	✓	+0.248	✓
H3	Spillover, 25%	+0.031	[−0.002, +0.067]	0.727		+0.053	✓
H3	Spillover, 50%	+0.034	[0.000, +0.078]	0.714	✓	+0.081	✓
H3	Serious spillover, 25%	+0.049	[0.000, +0.111]	0.662		+0.111	✓
H3	Serious spillover, 50%	+0.053	[0.000, +0.117]	0.648		+0.099	✓
H3	Late closure, 25%	+0.051	[−0.010, +0.125]	0.975		+0.032	
H3	Late closure, 50%	+0.053	[−0.013, +0.125]	0.975		+0.028	
H3	Seriously late, 25%	+0.067	[−0.008, +0.138]	0.937		+0.073	✓
H3	Seriously late, 50%	+0.093	[+0.007, +0.193]	0.622	✓	+0.027	
H4	Late closure	+0.043	[−0.013, +0.093]	0.975		+0.016	
H4	Seriously late	+0.045	[−0.020, +0.101]	0.975		+0.044	
H4	Spillover	+0.034	[−0.005, +0.082]	0.975		+0.089	✓
H4	Serious spillover	+0.088	[+0.018, +0.161]	0.223	✓	+0.178	✓
H4	Scope churn	+0.063	[−0.056, +0.207]	0.975		+0.065	

Table 4: Planning-time ROC-AUC, gradient boosting. Time: per-project time split. LOPO: leave one project out (no project feature).

	Time split			LOPO		
Outcome	scope	strict	+new	scope	strict	+new
Spillover	0.835	0.832	0.869	0.818	0.834	0.849
Serious spillover	0.681	0.752	0.769	0.609	0.723	0.748
Late closure	0.682	0.724	0.725	0.643	0.724	0.721
Seriously late	0.639	0.690	0.684	0.650	0.682	0.674
Scope churn	0.545	0.582	0.608	0.535	0.523	0.547

Table 5: In-sprint ROC-AUC at 25% and 50% of the planned duration, gradient boosting, time split.

Outcome	At	scope	worki	choet	+ctx	ours	+worki
Spillover	25%	0.831	0.785	0.885	0.897	0.916	0.914
	50%	0.834	0.743	0.903	0.910	0.937	0.937
Serious spillover	25%	0.695	0.665	0.794	0.828	0.843	0.841
	50%	0.688	0.708	0.833	0.851	0.886	0.886
Late closure	25%	0.678	0.571	0.665	0.724	0.716	0.717
	50%	0.676	0.634	0.679	0.742	0.731	0.736
Seriously late	25%	0.617	0.506	0.608	0.645	0.675	0.672
	50%	0.605	0.537	0.594	0.693	0.686	0.680

matter of how many issues were committed; serious spillover is mostly a matter of which team committed them. The same model also limits what history can do, which we saw only after the full run. If q_s is a property of the team and changes slowly, history separates teams but not the sprints of one team. Section 6.5 checks this. The median-based serious-spillover label shows the same pattern more clearly: strict+new scores 0.704 against 0.563 for scope.

AUC is high for spillover partly because the label is 90% positive. Only 73 nonempty test sprints finish everything they committed (110 for serious spillover). Average precision for finding those sprints (Table 13 in the Appendix) is several times the base rate but well below one, so the spillover models are much better at confirming the common outcome than at spotting the rare good sprint.

6.3 During the sprint

Table 5 gives the in-sprint AUC, on the nonempty sprints still open at the checkpoint (732 test sprints at 25%, 728 at 50%). Table 6 gives the fair comparisons.

Burndown shape adds nothing. Adding the worki set to ours changes AUC by at most 0.007 for any outcome at either checkpoint. On its own the worki set ranks worse than committed scope, with intervals below zero, for late closure at 25% (−0.107 [−0.164, −0.040]), for seriously late at both check-

Table 6: What each family adds to the others during the sprint: differences in AUC with 95% intervals over projects (gradient boosting, time split).

Outcome	At	Burndown shape added ours+worki − ours	Planning context added choet+ctx − choet	Our progress counts ours − choet+ctx
Spillover	25%	−0.002 [−0.006, +0.003]	+0.012 [−0.013, +0.043]	+0.018 [−0.008, +0.048]
	50%	0.000 [−0.006, +0.004]	+0.007 [−0.014, +0.032]	+0.027 [−0.004, +0.061]
Serious spillover	25%	−0.002 [−0.008, +0.004]	+0.034 [−0.011, +0.093]	+0.016 [−0.018, +0.048]
	50%	0.000 [−0.006, +0.005]	+0.018 [−0.021, +0.070]	+0.034 [+0.005, +0.069]
Late closure	25%	+0.001 [−0.009, +0.010]	+0.059 [−0.001, +0.125]	−0.008 [−0.022, +0.008]
	50%	+0.004 [−0.003, +0.012]	+0.063 [−0.001, +0.135]	−0.011 [−0.023, +0.003]
Seriously late	25%	−0.003 [−0.014, +0.008]	+0.038 [−0.029, +0.105]	+0.029 [+0.002, +0.055]
	50%	−0.007 [−0.018, +0.007]	+0.099 [+0.011, +0.192]	−0.006 [−0.042, +0.028]

Table 7: The binomial model as a predictor on the nonempty test sprints. AUC and calibration slope (1 is ideal).

	Spillover		Serious sp.	
Model	AUC	Slope	AUC	Slope
Gradient boosting, scope	0.835	0.89	0.681	0.88
Binomial, no learning	0.864	0.41	0.759	0.26
Beta-binomial, one parameter	0.854	0.59	0.745	0.49
Gradient boosting, strict+new	0.869	0.95	0.769	0.81

points and for spillover at 50%.

Planning context adds little once progress counts are in. Added to the choet counts, planning context changes AUC by 0.012 and 0.007 for spillover and 0.034 and 0.018 for serious spillover; none of these intervals clears zero. Under leave-one-project-out testing it helps serious spillover (0.057 [0.032, 0.084] at 25%). For seriously late at 50% it adds 0.099 [0.011, 0.192].

Our progress counts add a little beyond theirs for serious spillover at 50% (0.034 [0.005, 0.069]), and on the confirmation set at both checkpoints. The H3 gap is therefore partly progress counts and partly context, and neither part is large.

The spillover labels are where the signal is. Serious spillover goes from 0.769 at planning time to 0.843 at 25% and 0.886 at 50%. For late closure our set reaches only 0.716 and 0.731, against 0.678 and 0.676 for scope alone on the same sprints.

6.4 A model with no learning

Table 7 scores the binomial model of Section 3 as a predictor. For each sprint, q_s is the pooled completion rate of the team's last three finished sprints known at the start, or the training-set rate where there is no history. Nothing else is fitted. Its AUC is 0.864 against 0.869 for gradient boosting on spillover (difference 0.006 [−0.023, 0.048]) and 0.759 against 0.769 on serious spillover (0.010 [−0.034, 0.064]). The two cannot be told apart. For serious spillover the binomial model beats scope alone by 0.078 [0.011, 0.136]. On the confirmation set the gap to gradient boosting is wider (0.027 and 0.054, intervals [−0.030, 0.099] and [−0.004, 0.122]) and the binomial model still beats scope for serious spillover (0.124 [0.016, 0.224]). Most of what the learned model adds over scope at planning time is therefore an estimate of one number per team.

Its probabilities are another matter. The calibration slopes are 0.41 and 0.26, so the binomial model is far too sure of itself. Table 11 in the Appendix shows why. In every band of sprint size the observed variance of the carry ratio exceeds the binomial variance, by a factor of 1.7 for sprints of one to three issues and 5.8 for sprints of 16 or more. A gap that grows with sprint size is what Proposition 3 predicts when $r > 0$. The ratio of observed to binomial variance over the training sprints, the quantity behind Tarone's test of binomial fit [45], is 4.5, and the maximum-likelihood estimate is $r = 0.18$ (likelihood-ratio statistic 2,181 against $r = 0$). The beta-binomial model matches the observed variance for large sprints (0.041 against 0.044) and raises the slopes to 0.59 and 0.49, at a small cost in ranking (0.010 and 0.014 AUC). Neither is as well calibrated as gradient boosting (slopes 0.95 and 0.81). The learned model earns its keep through its probabilities far more than through its ranking.

6.5 Within projects

Every AUC so far is pooled over the test sprints of all 26 projects. A pair of sprints from two different projects counts as much as a pair from the same project, so a model can score well just by knowing which projects leave more work over. That is useful to someone who watches many teams. A team that uses a warning tool sees only its own sprints. Table 8 therefore repeats the main comparisons on the same models and test sprints, counting only pairs of sprints from the same project, with the same project-level bootstrap. We added this analysis after the full run.

The in-sprint findings hold. Progress counts beat burndown shape by 0.219 [0.102, 0.363] for spillover at 50%, and burndown shape still adds nothing. The new features keep their gain for spillover (0.040 [0.019, 0.068]). The history gain does not. For serious spillover it falls from 0.088 to 0.015 [−0.023, 0.088], and for late closure the history-aware set ranks a team's own sprints worse than scope alone (−0.035 [−0.052, −0.005]). This is the binomial reading again. The last three sprints estimate a team's completion rate, which separates teams well, but a rate that changes slowly says little about which of the team's next sprints will go badly. It also bears on the outside test of Section 6.7, which averages AUC within each outside project. Within TAWOS projects the history-aware set ranks serious spillover at 0.686 against 0.671 for scope, so the

Table 8: Pooled and within-project differences in AUC with 95% intervals over projects (gradient boosting, time split). Pooled: all test sprints, as in the rest of the paper. Within: only pairs of sprints from the same project. k: projects with both classes in the test set. Planning time unless a checkpoint is given.

Comparison	Outcome	Pooled	Within	k
New features	Spillover	+0.037 [+0.022, +0.056]	+0.040 [+0.019, +0.068]	22
	Serious spillover	+0.017 [−0.008, +0.048]	+0.012 [−0.030, +0.074]	23
History over scope	Spillover	+0.034 [−0.005, +0.082]	+0.016 [−0.010, +0.066]	22
	Serious spillover	+0.088 [+0.018, +0.161]	+0.015 [−0.023, +0.088]	23
	Serious sp. (median)	+0.141 [+0.065, +0.206]	+0.088 [−0.047, +0.218]	25
	Late closure	+0.043 [−0.013, +0.093]	−0.035 [−0.052, −0.005]	26
Progress over burndown	Spillover, 50%	+0.194 [+0.109, +0.272]	+0.219 [+0.102, +0.363]	21
	Serious spillover, 50%	+0.178 [+0.083, +0.261]	+0.152 [+0.044, +0.303]	23
Burndown added	Serious spillover, 50%	0.000 [−0.006, +0.005]	−0.006 [−0.019, +0.004]	23

outside test asked history for a gain it does not show at home either.

6.6 What a warning is worth

A difference of 0.03 AUC means little to a team. Table 12 in the Appendix asks a concrete question instead: if someone can look into only 10% or 20% of the sprints, how many of the ones flagged are real problems? The budgets are set over all test sprints, so this is the view of someone who oversees many teams, not of one team. For serious spillover in its median form, the history-aware model raises the hit rate among the top 10% from 75% to 81% (4 more real problems among 73 flagged sprints), against a base rate of 57%. For late closure the gain is 62% to 63%.

Table 9 applies the net benefit (9) on the 728 sprints scored at all three stages. Never warning scores zero. For serious spillover, warning on every sprint is almost as good as any model unless warnings are costly: at $p^* = 0.5$ warning on all sprints gives 0.352 against 0.353 at planning and 0.380 at 50%. Only at $p^* = 0.9$, where a warning must be nearly certain to pay, does the model clearly add value (0.022 at planning and 0.042 at 50%, against −0.048 for warning on all). There, waiting until 50% of the sprint pays only if a warning at that point still averts at least 0.51 of what an early warning would. For late closure the picture is reversed: warning on all sprints loses at any threshold above the base rate, the planning-time model gains a little (0.099 at $p^* = 0.3$, 0.032 at 0.5), and the in-sprint models gain no more than the planning model, so waiting never pays. Fig. 3 shows the calibration behind these numbers.

Table 9: Net benefit (9) per sprint, in units of the averted failure cost, on the same 728 test sprints. All: warn on every sprint. Never warning scores 0.

Outcome	p^*	All	Planning	25%	50%
Serious spillover	0.3	+0.552	+0.550	+0.556	+0.568
Serious spillover	0.5	+0.352	+0.353	+0.369	+0.380
Serious spillover	0.7	+0.152	+0.177	+0.187	+0.201
Serious spillover	0.9	−0.048	+0.022	+0.037	+0.042
Late closure	0.3	+0.045	+0.099	+0.088	+0.093
Late closure	0.5	−0.155	+0.032	+0.032	+0.033
Late closure	0.7	−0.355	−0.004	−0.009	−0.003
Late closure	0.9	−0.555	−0.002	−0.003	−0.004

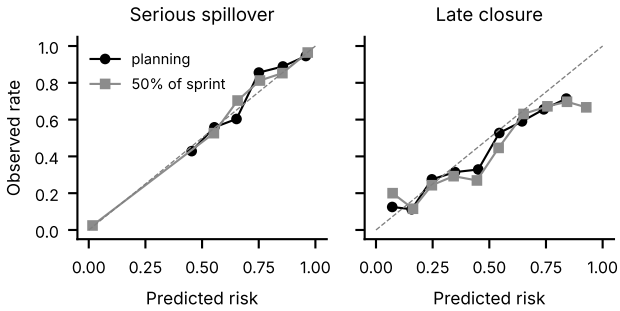


Figure 3: Reliability of the gradient-boosting risk on the sprints of Table 9: mean predicted risk against the observed rate, in bins of width 0.1 with at least ten sprints.

6.7 Outside projects

Table 10 trains on all of TAWOS and tests on each outside project with at least five sprints in each class. We find no evidence of transfer. For the two spillover labels, where history helped most inside TAWOS, scope alone scores 0.778 and 0.594, and the history-aware sets score lower. For late closure all three sets sit between 0.641 and 0.666. Only 4 outside projects are testable for spillover, and the spread across projects is large, so we can only say that we see no transfer, not that there is none. We cannot tell whether the cause is the new organisations, because two simpler reasons could explain it. The outside test scores ranking within each project, where history adds little even inside TAWOS (Section 6.5), and its feature sets leave out story points, which MariaDB does not record.

6.8 Robustness

Table 14 in the Appendix repeats the main planning-time comparisons under five variants. *Tuning:* with a small grid chosen on a time-ordered validation slice, the new features add 0.028 [0.009, 0.042] for spillover and history beats scope for serious spillover by 0.081 [0.023, 0.149]. *A single calendar cut* in early 2018 (train on sprints whose outcome was known before it, test on sprints that start after it, 20 projects): 0.035 [0.018, 0.056] and 0.079 [0.005, 0.149]. *Without Appcelerator*

Table 10: Outside projects: mean ± s.d. of per-project AUC, gradient boosting trained on all TAWOS sprints. k: projects with at least five sprints in each class.

Outcome	k	scope	strict	strict+new
Spillover	4	0.778 ± 0.138	0.759 ± 0.142	0.755 ± 0.152
Serious spillover	9	0.594 ± 0.122	0.559 ± 0.129	0.580 ± 0.110
Late closure	8	0.666 ± 0.144	0.641 ± 0.129	0.652 ± 0.104
Seriously late	8	0.564 ± 0.160	0.584 ± 0.211	0.606 ± 0.227
Scope churn	5	0.428 ± 0.209	0.596 ± 0.093	0.609 ± 0.140

Daemon and Hyperledger Indy Node, the two projects whose sprints mostly finish nothing although they pass the measurability check: 0.038 [0.023, 0.057] and 0.071 [0.003, 0.146]. *Before the measurability check*, on all 29 projects, the new features added more for serious spillover and less for spillover, which is why we report the check as a contribution. *Sprint-history coverage:* on the 15 projects where at least 85% of issues ever placed in a sprint have a sprint history, the new features add 0.038 [0.011, 0.059] for spillover and history beats scope for serious spillover by 0.142 [0.041, 0.228]. Burndown shape added to ours changes AUC by at most 0.011, and planning context added to the choet counts gains more than on the full cohort (0.044 [0.018, 0.083] for spillover at 25%), so the weak planning-context result may partly reflect missing sprint history rather than a weak signal. *Story-point provenance:* on test sprints with no story-point fallback, the new features add 0.041 [0.015, 0.064] for spillover and history beats scope for serious spillover by 0.125 [0.025, 0.215]. The five time-ordered repeats agree with the time split (strict+new scores 0.858 ± 0.019 for spillover, 0.767 ± 0.027 for serious spillover and 0.727 ± 0.013 for late closure). Calibration is in Table 15: slopes from 0.81 to 0.97 for the spillover labels and from 0.68 to 0.79 for the late labels, which are overconfident.

7 Discussion

7.1 What a team can take from this

During a sprint, simple progress counts (how much is done, how much is left, how fast it is going) carry nearly all of the signal, within a team as well as across teams, and their warnings for leftover work are close to calibrated. Burndown-shape indicators add nothing on top of them.

Before a sprint starts, the picture depends on who is looking. Someone who oversees many teams learns most from each team's recent completion rate, and the binomial model gives that ranking with nothing learned, though its probabilities need the beta-binomial correction or a learned model before anyone acts on them. A single team learns little from its own history that the size of the sprint does not already say. The new features help a little with plain spillover, but that gain did not hold on the confirmation set.

We see no sign that a model trained on other organisations' teams does better than a size rule, and a threshold carried from one team to another changes what a miss is worth (Proposition 2). For outcomes as common as spillover, the decision analysis adds a further caution: unless a warning is costly, warning on every sprint is nearly as good as any model.

7.2 Why lateness is hard

Late closure is the moment someone pressed "Complete sprint" in Jira. That depends on meetings, holidays and habit as much as on the work, and no feature family we tried moves it much. We keep it for continuity, but we would not build a tool on it. Seriously late is no easier to rank, and its scores are among the worst calibrated in the study.

7.3 Threats to validity

Construct. The labels come from Jira fields, not from asking teams whether a sprint went badly. Late closure and seriously late measure when a sprint was closed in the tool. Spillover counts issues, not effort. The median nonempty sprint leaves 67% of its committed issues unresolved at the planned end, and 26% of nonempty sprints resolve none. In open-source projects an issue is often resolved at a release or a merge rather than when the work stops, so the carry ratio partly measures when a project resolves issues, not only how much work was left. The measurability check catches only issues closed with no date at all; a project that sets resolution dates late passes it and still overstates spillover. We think this is the largest threat to the spillover labels, and only the teams themselves can settle it.

Internal. The reconstruction depends on the change log. Where it is missing, story points fall back on the exported value; we measured the share and repeated the main comparisons without it. Issue type, priority and assignee are left out because only their final values exist, which also means the choet set lacks team and issue features its authors used. The strict, choet+ctx and ours sets carry the project as a feature; leave-one-project-out testing removes it.

Hypotheses and analysis choices. The hypotheses were written after a development run on eight projects. Several choices were made after the full run: the measurability check, the choet+ctx set, the Holm adjustment, the confirmation set, the tuned and calendar-cut variants, the within-project analysis and the decision analysis. All are logged with dates and reasons, and we report their effect whichever way it went. The confirmation set is our main answer to this threat: the in-sprint and serious-spillover results hold there, the new-features result does not. It is not an independent replication, though. Its 19 projects are part of the full cohort, so the two estimates share 1,538 sprints, and every choice made after the full run was made with those projects in view.

Conclusion. The project-level interval resamples only 26 clusters, so it is wide. We treat a result as holding only when the project interval clears zero, and report the adjusted values beside it.

External. All projects are open source, and a handful of organisations contribute most TAWOS projects. The outside test covers two more organisations and ten projects, few of which have enough sprints in both classes for every label.

8 Conclusion

Under one leakage-aware protocol, and after checking that the outcomes can be measured, the kinds of signal used in the sprint-risk literature differ sharply. Progress counts dominate during a sprint and burndown shape adds nothing. Team history tells teams apart, but adds little to ranking one team's own sprints and nothing for when a sprint is closed, and we see no sign that it helps on another organisation's projects. Because every AUC in this literature is pooled over projects, results like ours should be reported within projects too. The formal setting gives simple reasons for these patterns: empty sprints inflate AUC by an exact amount, a fitted threshold carries a hidden cost ratio, the choice between "any" and "serious" spillover decides whether history can matter, a completion rate that belongs to the team says little about which of that team's sprints will go badly, and issues that do not finish independently explain why a model with no learning ranks well but cannot give trustworthy probabilities. The obvious next step is to ask teams whether the sprints these models flag are the ones they would have wanted to hear about.

Data Availability

Code, export queries, the measurability and reconstruction checks, the reconstructed frames, every result file and the dated study plan with its change log are at `https://github.com/hani-ziad/journal-ieee-access-2027`. TAWOS [10] is released under the Apache License 2.0 for research use. The repository includes the eight-project extract used for development and the SQL that makes the full extract from the public TAWOS database; the full extract is too large for the repository and is available from the author. The outside projects' issues come from the public Jira sites of MariaDB and MongoDB; we do not redistribute them, but the repository includes the scripts that pull and convert them.

Appendix: Further Results

Table 11: Variance of the carry ratio around $1 - \hat{q}_s$ on the training sprints, by committed issues: observed, binomial (6) and beta-binomial (7) with $r = 0.18$.

Issues	Sprints	Observed	Binomial	Beta-bin.
1–3	471	0.201	0.120	0.132
4–7	373	0.094	0.038	0.064
8–15	396	0.064	0.017	0.047
16+	435	0.044	0.008	0.041

Table 12: Warning budgets at planning time: share of flagged sprints that are real problems (%) when the top 10% or 20% of test sprints by risk are flagged. Base: positive rate.

Outcome	Flag	Base	scope	strict	+new
Serious sp. (median)	10%	57	75	78	81
	20%	57	68	71	75
Late closure	10%	35	62	64	63
	20%	35	65	64	63

Table 13: Average precision for finding the sprints that do *not* spill (planning time, gradient boosting, time split). Base: share of such sprints among the nonempty test sprints.

Outcome	n	Base	scope	strict	+new
Spillover	73	0.099	0.378	0.464	0.499
Serious spillover	110	0.150	0.315	0.407	0.400

References

[1] M. Choetkiertikul, H. K. Dam, T. Tran, A. Ghose, and J. Grundy, "Predicting delivery capability in iterative software development," *IEEE Trans. Softw. Eng.*, vol. 44, no. 6, pp. 551–573, Jun. 2018, doi: 10.1109/TSE.2017.2693989.

[2] M. Choetkiertikul, P. Banyongrakkul, C. Ragkhitwetsagul, S. Tuarob, H. K. Dam, and T. Sunetnanta, "Sprint2Vec: A deep characterization of sprints in iterative software development," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 220–242, Jan. 2025, doi: 10.1109/TSE.2024.3509016.

[3] P. Hearty, N. Fenton, D. Marquez, and M. Neil, "Predicting project velocity in XP using a learning dynamic Bayesian network model," *IEEE Trans. Softw. Eng.*, vol. 35, no. 1, pp. 124–137, Jan. 2009, doi: 10.1109/TSE.2008.76.

[4] Y. J. Pérez Castillo, S. D. Orantes Jiménez, and P. O. Letelier Torres, "Sprint management in Agile approach: Progress and velocity evaluation applying machine learning," *Information*, vol. 15, no. 11, Art. no. 726, 2024, doi: 10.3390/info15110726.

[5] P. G. Obike, V. E. Ekong, and O. U. Obot, "A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning," *Int. J. Comput. Appl.*, vol. 187, no. 49, pp. 40–53, 2025, doi: 10.5120/ijca2025925837.

[6] E. Kula, E. Greuter, A. van Deursen, and G. Gousios, "Factors affecting on-time delivery in large-scale agile software development," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3573–3592, Sep. 2022, doi: 10.1109/TSE.2021.3101192.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012, doi: 10.1145/2382577.2382579.

Table 14: Main planning-time differences in AUC under each robustness variant. Columns: new features (strict+new − strict) and history over scope (strict+new − scope) for spillover (Sp), serious spillover (Ss) and late closure (Lc). *: project-level interval above zero.

	New features			History over scope		
Variant	Sp	Ss	Lc	Sp	Ss	Lc
Primary	+0.037*	+0.017	+0.001	+0.034	+0.088*	+0.043
Tuned	+0.028*	+0.024*	−0.002	+0.027*	+0.081*	+0.021
Calendar cut	+0.035*	+0.021	−0.003	+0.053*	+0.079*	+0.032
Without 2 projects	+0.038*	+0.018	+0.001	+0.031	+0.071*	+0.042
Uncleaned (29)	+0.028*	+0.026*	−0.006	+0.040*	+0.104*	+0.011

Table 15: Calibration on the time-split test set, gradient boosting. Ref.: Brier score of the test prevalence. ECE: expected calibration error, ten bins.

Outcome	Stage	Brier	Ref.	ECE	Slope
Spillover	planning	0.066	0.090	0.028	0.95
	in-sprint 25%	0.053	0.090	0.014	0.97
	in-sprint 50%	0.043	0.089	0.010	0.95
Serious spillover	planning	0.109	0.127	0.036	0.81
	in-sprint 25%	0.089	0.127	0.020	0.87
	in-sprint 50%	0.070	0.126	0.013	0.93
Late closure	planning	0.198	0.226	0.060	0.79
	in-sprint 25%	0.200	0.225	0.059	0.73
	in-sprint 50%	0.196	0.226	0.069	0.78
Seriously late	planning	0.147	0.155	0.055	0.68
	in-sprint 25%	0.139	0.146	0.050	0.69
	in-sprint 50%	0.140	0.147	0.026	0.70

[8] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[9] H. AbuSharkh, M. Hanif, and R. Damseh, "Reconstructing sprint-start Jira state for leakage-aware outcome prediction," submitted to the IEEE Canadian Conf. Electr. Comput. Eng. (CCECE), 2027.

[10] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, "A versatile dataset of agile open source software projects," in *Proc. 19th Int. Conf. Mining Softw. Repositories (MSR)*, 2022, pp. 707–711, doi: 10.1145/3524842.3528029.

[11] M. Choetkiertikul, H. K. Dam, T. Tran, and A. Ghose, "Characterization and prediction of issue-related risks in software projects," in *Proc. IEEE/ACM 12th Working Conf. Mining Softw. Repositories (MSR)*, 2015, pp. 280–291, doi: 10.1109/MSR.2015.33.

[12] M. Choetkiertikul, H. K. Dam, T. Tran, and A. Ghose, "Predicting the delay of issues with due dates in software projects," *Empir. Softw. Eng.*, vol. 22, no. 3, pp. 1223–1263, 2017, doi: 10.1007/s10664-016-9496-7.

[13] E. Kula, A. van Deursen, and G. Gousios, "Modeling team dynamics for the characterization and prediction of delays in user stories," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2021, pp. 991–1002, doi: 10.1109/ASE51524.2021.9678939.

[14] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for estimating story points," *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 637–656, Jul. 2019, doi: 10.1109/TSE.2018.2792473.

[15] S. Porru, A. Murgia, S. Demeyer, M. Marchesi, and R. Tonelli, "Estimating story points from issue reports," in *Proc. 12th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE)*, 2016, pp. 1–10, doi: 10.1145/2972958.2972959.

[16] M. Fu and C. Tantithamthavorn, "GPT2SP: A transformer-based agile story point estimation approach," *IEEE Trans. Softw. Eng.*, vol. 49, no. 2, pp. 611–625, Feb. 2023, doi: 10.1109/TSE.2022.3158252.

[17] V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2677–2697, Apr. 2023, doi: 10.1109/TSE.2022.3228739.

[18] M. Ortu, G. Destefanis, B. Adams, A. Murgia, M. Marchesi, and R. Tonelli, "The JIRA repository dataset: Understanding social aspects of software development," in *Proc. 11th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE)*, 2015, pp. 1–4, doi: 10.1145/2810146.2810147.

[19] L. Montgomery, C. Lüders, and W. Maalej, "An alternative issue tracking dataset of public Jira repositories," in *Proc. 19th Int. Conf. Mining Softw. Repositories (MSR)*, 2022, pp. 73–77, doi: 10.1145/3524842.3528486.

[20] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 43, no. 1, pp. 1–18, Jan. 2017, doi: 10.1109/TSE.2016.2584050.

[21] A. A. Bangash, H. Sahar, A. Hindle, and K. Ali, "On the time-based conclusion stability of cross-project defect prediction models," *Empir. Softw. Eng.*, vol. 25, no. 6, pp. 5047–5083, 2020, doi: 10.1007/s10664-020-09878-9.

[22] D. Falessi, J. Huang, L. Narayana, J. F. Thai, and B. Turhan, "On the need of preserving order of data when validating within-project defect classifiers," *Empir. Softw. Eng.*, vol. 25, no. 6, pp. 4805–4830, 2020, doi: 10.1007/s10664-020-09868-x.

[23] M. Jimenez, R. Rwemalika, M. Papadakis, F. Sarro, Y. Le Traon, and M. Harman, "The importance of accounting for real-world labelling when predicting software vulnerabilities," in *Proc. 27th ACM Joint Meeting Eur. Softw.*

Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), 2019, pp. 695–705, doi: 10.1145/3338906.3338941.

[24] M. Shepperd, D. Bowes, and T. Hall, "Researcher bias: The use of machine learning in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 40, no. 6, pp. 603–616, Jun. 2014, doi: 10.1109/TSE.2014.2322358.

[25] A. E. Márquez-Chamorro, M. Resinas, and A. Ruiz-Cortés, "Predictive monitoring of business processes: A survey," *IEEE Trans. Serv. Comput.*, vol. 11, no. 6, pp. 962–977, 2018, doi: 10.1109/TSC.2017.2772256.

[26] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, "Outcome-oriented predictive process monitoring: Review and benchmark," *ACM Trans. Knowl. Discov. Data*, vol. 13, no. 2, pp. 1–57, 2019, doi: 10.1145/3301300.

[27] C. Di Francescomarino, C. Ghidini, F. M. Maggi, and F. Milani, "Predictive process monitoring methods: Which one suits me best?" in *Business Process Management (BPM)*, Lecture Notes in Computer Science, 2018, pp. 462–479, doi: 10.1007/978-3-319-98648-7_27.

[28] S. A. Fahrenkrog-Petersen, N. Tax, I. Teinemaa, M. Dumas, M. de Leoni, F. M. Maggi, and M. Weidlich, "Fire now, fire later: Alarm-based systems for prescriptive process monitoring," *Knowl. Inf. Syst.*, vol. 64, no. 2, pp. 559–587, 2022, doi: 10.1007/s10115-021-01633-w.

[29] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[30] D. J. Hand, "Measuring classifier performance: A coherent alternative to the area under the ROC curve," *Mach. Learn.*, vol. 77, no. 1, pp. 103–123, 2009, doi: 10.1007/s10994-009-5119-5.

[31] J. Hernández-Orallo, P. Flach, and C. Ferri, "A unified view of performance metrics: Translating threshold choice into expected classification loss," *J. Mach. Learn. Res.*, vol. 13, no. 91, pp. 2813–2869, 2012.

[32] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2001, pp. 973–978.

[33] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Rev.*, vol. 78, no. 1, pp. 1–3, 1950, doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[34] B. Van Calster, D. J. McLernon, M. van Smeden, L. Wynants, and E. W. Steyerberg, "Calibration: The Achilles heel of predictive analytics," *BMC Med.*, vol. 17, 2019, doi: 10.1186/s12916-019-1466-7.

[35] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.

[36] J. G. Skellam, "A probability distribution derived from the binomial distribution by regarding the probability of success as variable between the sets of trials," *J. R. Stat. Soc. Ser. B*, vol. 10, no. 2, pp. 257–261, 1948, doi: 10.1111/j.2517-6161.1948.tb00014.x.

[37] A. J. Vickers and E. B. Elkin, "Decision curve analysis: A novel method for evaluating prediction models," *Med. Decis. Making*, vol. 26, no. 6, pp. 565–574, 2006, doi: 10.1177/0272989X06295361.

[38] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[39] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[40] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, no. 85, pp. 2825–2830, 2011.

[41] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *J. R. Stat. Soc. Ser. B*, vol. 69, no. 3, pp. 369–390, 2007, doi: 10.1111/j.1467-9868.2007.00593.x.

[42] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[43] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 233–240, doi: 10.1145/1143844.1143874.

[44] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950, doi: 10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3.

[45] R. E. Tarone, "Testing the goodness of fit of the binomial distribution," *Biometrika*, vol. 66, no. 3, pp. 585–590, 1979, doi: 10.1093/biomet/66.3.585.